Exploiter les fonctionnalités des bases de données NoSQL MongoDB

Exploiter les fonctionnalités des bases de données NoSQL MongoDB

Découvrir les bases de données NoSQL

1. Qu'est-ce qu'une base de données NoSQL ?

a) Du SGBD relationnel au NoSQL

Les SGBD relationnels (SQL) gèrent des données bien structurées, organisées en tables :

- Colonnes → propriétés
- Lignes → données

Avantages des SGBD relationnels :

- Séparation logique et physique
- Forte structuration des données
- Typage strict
- Représentation tabulaire
- Langage déclaratif (SQL)
- Intégrité des données via contraintes
- Cohérence transactionnelle (ACID)

Limites face aux 3V du Big Data :

V	Limite SQL
Volume	Trop de données pour un seul serveur
Velocity	Difficulté à traiter rapidement de gros volumes

Formateur : AARAB HAFID

V	Limite SQL
Veracity	Rigidité du schéma, problème avec données semi-structurées ou non-structurées

Exemple :

Imaginez une agence de voyage avec 1 million de clients et 10 millions de réservations par an. Les tables SQL classiques deviennent difficiles à gérer rapidement.

b) Définition du NoSQL

NoSQL = "Not Only SQL"

- Nouvelle manière de stocker et gérer les données, non obligatoire relationnelle
- Supporte des types variés : XML, JSON, collections d'objets, graphes…
- Permet la distribution des données sur plusieurs serveurs

Complète SQL pour :

- **Big Data**
- Données semi-structurées ou non-structurées
- Applications nécessitant scalabilité horizontale

c) Historique

Mouvement apparu milieu années 2000 pour répondre aux besoins Big Data des GAFA :

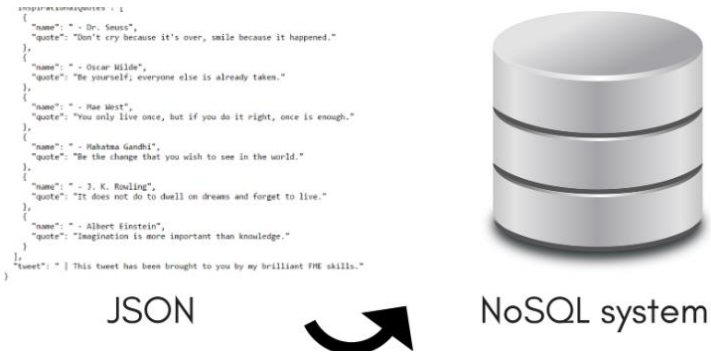
Entreprise	Base NoSQL
Google	HBase
Amazon	DynamoDB
Facebook	Cassandra
Apple	Base interne

d) Avantages du NoSQL

- Format clé-valeur simple à manipuler
- Stockage direct d'objets/langages (JSON, listes…)
- Open-source, pas de licence coûteuse
- Scalabilité facile : ajout de serveurs
- Données regroupées par unités logiques → moins de jointures

Exemple :

Pour obtenir un client et ses réservations : **pas besoin de jointures SQL**



e) Inconvénients du NoSQL

- Pas de clé étrangère → cohérence gérée côté application
- Non adapté aux transactions très sécurisées (banque)
- SQL et NoSQL non compatibles

2. Comparaison SQL vs NoSQL

Critère	SQL	NoSQL
Structure	Tables avec schéma strict	Collections avec schéma libre
Scalabilité	Verticale (CPU, RAM, SSD)	Horizontale (ajout de serveurs)
Transactions	ACID(Atomicité, Cohérence, Isolation, Durabilité)	BASE (Basically Available, Soft State, Eventually Consistent)
Performance	Bonne pour volumes limités	Optimisé pour Big Data et haut volume
Cas d'usage	Applications transactionnelles, bancaires	Applications web, réseaux sociaux, données massives
Requêtes	SQL standard	Langages propres (MongoDB Query, CQL…)

- **SQL → fiable et cohérent, limité pour gros volumes**
- **NoSQL → rapide et scalable, nécessite gestion applicative de la cohérence**

	SQL	NoSQL
Type	Relationnelle	Non-Relationnelle
Données	Données structurées	Données non-structurées
Schéma	Statique	Dynamique
Scalabilité	Verticale	Horizontale
Langage	Structured Query Language	Un-structured Query language
Flexibilité	Rigide	Flexible
Support	Support éditeurs	Support communauté Open-Source

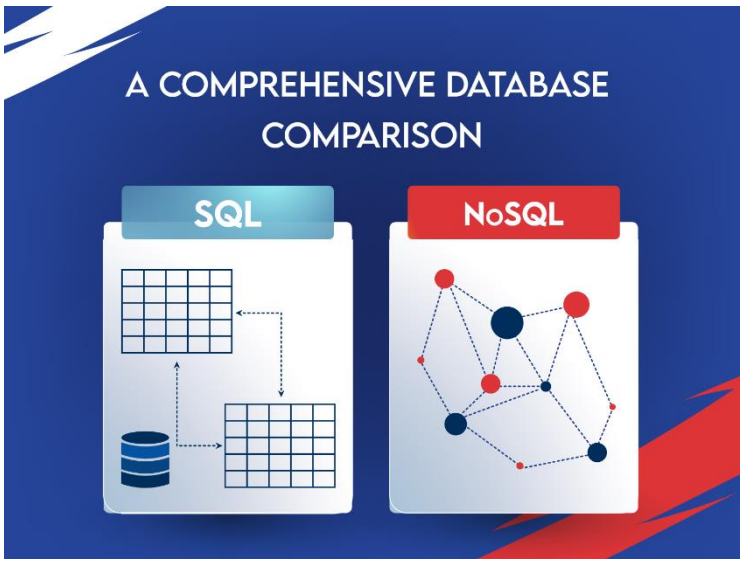
3. Caractéristiques des bases NoSQL

a) Modèle sans schéma (Schema Free)

- Données dénormalisées, documents imbriqués
- Structure modifiable facilement au fil du temps

Exemple :

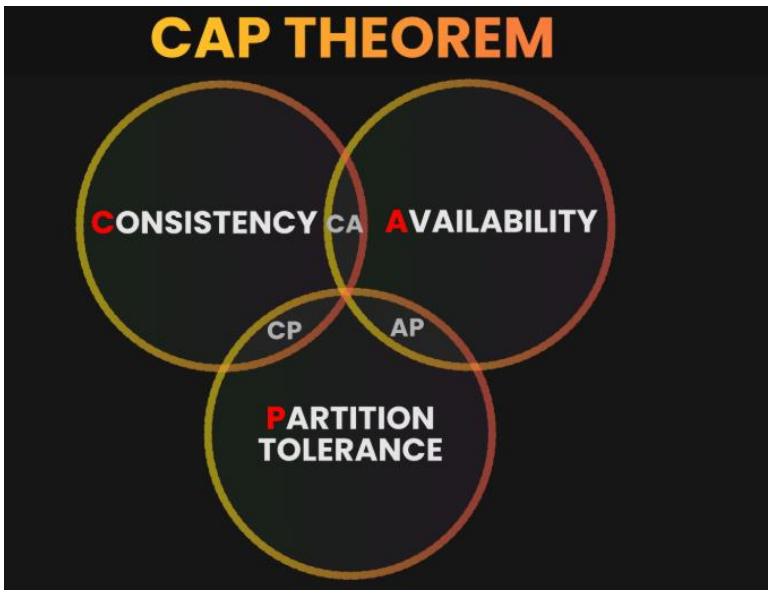
```
{
  "client": "Sanaa",
  "voyages": [
    {"destination": "Paris", "date": "2025-12-20"},
    {"destination": "Rome", "date": "2026-01-15"}
  ]
}
```



b) Architecture distribuée

- Données et traitements répartis sur plusieurs serveurs
- Partitionnement horizontal, tolérance aux pannes

Théorème CAP :



Lettre	Signification
C	Cohérence
A	Disponibilité
P	Tolérance au partitionnement

Dans NoSQL, on privilégie souvent AP (Disponibilité + Tolérance) plutôt que CP.

c) BASE vs ACID (explication complète)

ACID(Atomicité, Cohérence, Isolation, Durabilité) (SQL) : transactions fiables et cohérentes

- A = Atomicité : tout ou rien
- C = Cohérence : base toujours valide
- I = Isolation : transactions concurrentes séparées
- D = Durabilité : données permanentes après validation

BASE (NoSQL) : performance et disponibilité

- B = Basically Available : données toujours disponibles
- S = Soft State : état de la base peut changer temporairement
- E = Eventually Consistent : cohérence progressive

Exemple : agence de voyage

Scénario	ACID	BASE
Réservation vol Paris	Atomicité : tout ou rien	Peut apparaître d'abord sur un serveur, propagation progressive
Clients multiples réservent même vol	Isolation : pas de conflit	Conflits résolus progressivement, priorité à disponibilité

Scénario	ACID	BASE
Serveur tombe après confirmation	Durabilité : données conservées	Disponibilité garantie via autres serveurs

ACID (bases SQL)

Objectif : ZÉRO erreur, ZÉRO incohérence

Imagine une **banque**

Atomicité (tout ou rien)

Définition simple

Une opération est **indivisible**.

Soit tout réussit, soit rien n'est enregistré.

Exemple

Transfert bancaire :

- Retirer 100 DH du compte A
- Ajouter 100 DH au compte B

Si les deux réussissent → OK
Si une seule réussit → TOUT est annulé

Jamais d'argent perdu ou créé par erreur

Cohérence (base toujours correcte)

Définition

Après chaque opération, **les règles sont respectées**.

Exemple

- Solde d'un compte **ne peut pas être négatif**
- Une commande doit avoir **un client existant**

Si une règle est violée → opération refusée

La base ne devient jamais "fausse"

Isolation (chacun son tour)

Définition

Plusieurs utilisateurs peuvent travailler **en même temps**,
mais **sans se perturber**.

Exemple

Deux clients achètent **le dernier billet d'avion**

- Client A achète
- Client B arrive en même temps

La base **ne vend pas deux fois le même billet**

Durabilité (données jamais perdues)

Définition

Une fois validée, **l'opération est définitive**.

Exemple

- Paiement confirmé
- Coupure d'électricité

Les données sont **toujours là au redémarrage**

BASE (bases NoSQL)

Objectif : RAPIDITÉ et DISPONIBILITÉ

Imagine maintenant **Facebook, Google, Booking**
Des millions d'utilisateurs **en même temps**.

Basically Available (toujours accessible)

Définition

Le système **répond toujours**, même s'il y a un problème.

Exemple

- Un serveur tombe en panne
- L'application continue à fonctionner

Mieux vaut **une info approximative que pas d'info du tout**

Soft State (état temporairement instable)

Définition

Les données peuvent être **temporairement différentes** selon les serveurs.

Exemple

- Tu modifies ton profil
- Un ami voit l'ancienne version pendant quelques secondes

Ce n'est pas grave

Eventually Consistent (cohérence plus tard)

Définition

Les données deviennent **correctes avec le temps**.

Exemple

- Tu réserves un voyage
- Tous les serveurs se mettent à jour progressivement

Après un moment, **tout est synchronisé**

d) Manipulation de données complexes

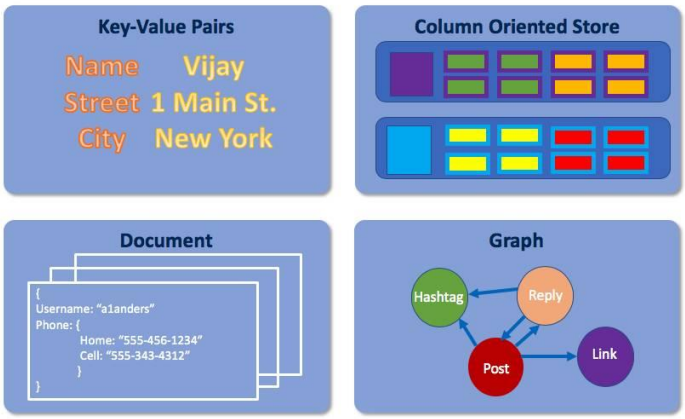
- Documents imbriqués, collections, tableaux
- Pas besoin de jointures multiples

4. Types de bases NoSQL

Type	Description	Exemples	Usage
Clé-Valeur	Paires clé/valeur	DynamoDB, Berkeley DB, Voldemort	Profils utilisateurs, caches
Document	Documents JSON/XML	MongoDB, CouchDB, Cassandra	Gestion agences, commandes clients

Type	Description	Exemples	Usage
Colonne	Données par colonnes	BigTable, HBase, SparkSQL	Analytics, gros volumes
Graphe	Basé sur graphes	Neo4j	Réseaux sociaux, relations complexes

5. Comparaison entre types NoSQL



Type	Coût	Cohérence	Disponibilité	Langage interrogation	Usage
Clé/valeur	+	++	++	++	Profil utilisateur, cache
Document	++	+	+	++	Applications web, stockage cloud
Colonne	++	+	+	+	Analytique, gros volumes
Graphe	++	++	+	++	Réseaux sociaux, relations complexes

6. Pourquoi choisir MongoDB ?

- Nom : Humongous DB → grande capacité
- **Avantages :**
  - Open-source
  - Orientée document
  - Haute performance et disponibilité
  - Scalabilité automatique
  - Respect du théorème CAP (CP)
  - Interface facile : Java, JS, PHP…
  - Multi-plateforme (VM, cloud)

Cas concret : stocker informations clients et voyages dans un seul document JSON, simplifiant récupération et modification.

7. Exemple

Document MongoDB pour agence de voyage :

```
{
  "client": "Amine",
  "email": "amine@example.com",
  "voyages": [
    {"destination": "Paris", "date": "2025-12-20"},
    {"destination": "Tokyo", "date": "2026-01-15"}
  ]
}
```

Requête pour récupérer les voyages d'Amine :

db.clients.find({"client": "Amine"})

-----Avantages : tout est dans un seul document → pas de jointures